



INTERNATIONAL CYBERSECURITY AND DIGITAL FORENSICS ACADEMY

ASSIGNMENT

Assignment Title: PC Systems Fundamentals, Disk Sectors and Partition Analysis

Student Name: Ahmad Zubairu Garba

Student ID: 2025/FWSD/11463

Course Title: Basic Computer Skills for Digital Forensics

Course Code: SBT-DF201

Instructor: Aminu Idris

August 09, 2026

INTRODUCTION

This comprehensive digital forensics laboratory exercise is meticulously designed to establish a profound, foundational understanding of computer architecture by systematically exploring core investigative domains, including system identification, multi-layered evidence mapping, real-time process observation, and detailed disk structure enumeration.

Students and investigators learn how to safely, methodically, and legally inspect target operational systems without altering, contaminating, or compromising volatile or persistent digital evidence.

By leveraging specialized read-only utilities, forensic examiners bridge the gap between theoretical system architecture and practical field triage across both Microsoft Windows and Linux-based operational ecosystems.

Analyzing underlying partition tables, tracking exact drive capacities, dissecting boot mechanisms, and auditing running process trees equips practitioners with essential technical acumen.

These core competencies form the vital structural baseline required to execute advanced digital investigations, incident response protocols, and complex malware analysis workflows effectively.

Tools and Environments Used

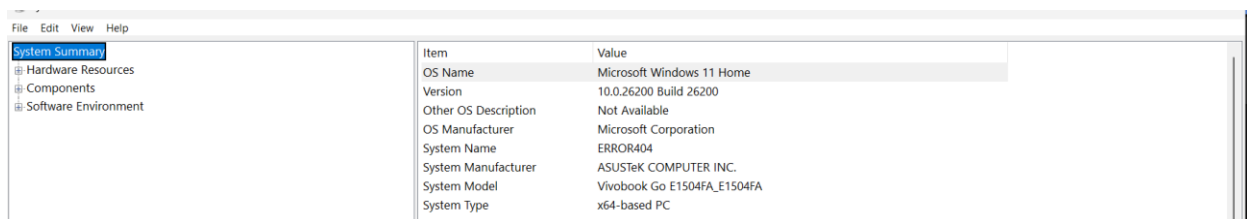
- Operating Systems:
 - Microsoft Windows 11 Home serves as the primary proprietary desktop environment for examining standard user configurations, registry structures, and native GUI-based administrative tools.
 - Kali Linux acts as the dedicated open-source forensic and penetration testing platform utilized for low-level disk analysis, command-line triage, and advanced packet evaluation.
- Windows Utilities & Commands:
 - System Information (msinfo32) provides a comprehensive hardware, system components, and software environment overview.
 - Windows Task Manager enables real-time observation of active applications, running processes, performance metrics, and resource utilization.
 - PowerShell cmdlets specifically Get-Disk, Get-Partition, and Get-Volume—facilitate precise, scriptable storage enumeration alongside tasklist for process auditing.
 - Disk Management (diskmgmt.msc) offers a graphical partition mapping interface, while the Command Prompt utility diskpart delivers deep, text-based disk partitioning and volume management controls.
- Linux Utilities & Commands:
 - lsblk lists block devices in a clean, hierarchical tree format to visualize drive layouts and mount points.
 - fdisk and parted are employed for inspecting, analyzing, and manipulating partition tables across various disk label formats.
 - blkid locates and prints block device attributes, including UUIDs and filesystem types, for precise evidence mapping.
 - df reports file system disk space usage to track available capacity and volume saturation during triage.

Part A - System Identification and Evidence Layer Mapping

To establish a foundational profile of the target system, system specifications and storage properties were captured via safe, read-only utilities, and evidence sources were mapped across computer system layers.

The system hardware and configuration details extracted from System Information include:

- **OS Name & Version:** Microsoft Windows 11 Home, Version 10.0.26200 Build 26200.
- **System Type:** x64-based PC powered by an AMD Ryzen 5 7520U with Radeon Graphics.
- **BIOS Mode & Security:** UEFI firmware with Secure Boot configured to Off.
- **Physical Memory:** Installed physical memory and system layout details as documented from the host diagnostics.



The screenshot shows the Windows System Information application. The left sidebar has a tree view with 'System Summary' selected. The main pane displays a table of system information.

Item	Value
OS Name	Microsoft Windows 11 Home
Version	10.0.26200 Build 26200
Other OS Description	Not Available
OS Manufacturer	Microsoft Corporation
System Name	ERROR404
System Manufacturer	ASUSTeK COMPUTER INC.
System Model	Vivobook Go E1504FA_E1504FA
System Type	x64-based PC

(System Information window displaying OS Name, Version, System Model, Processor, BIOS Mode, and Secure Boot State)

To examine storage configuration safely without modifying persistent data, read-only PowerShell inspection commands were executed, revealing the multi-partition layout and file system metrics:

- **Physical Disk & Model:** The storage configuration runs on the host NVMe hardware with healthy operational status.
- **Partition Architecture:** The partition table contains multiple segments including the EFI system partition, reserved spaces, recovery volumes, and the primary OS partition.
- **File System Type:** Volume inspection confirms the primary drives utilize the NTFS file system, providing necessary transactional logging and metadata support for forensic investigation.

```
Windows PowerShell
PS C:\WINDOWS\System32> Get-Disk

Number FriendlyName SerialNumber HealthStatus
-----
0 S 0025_38EB_3170_18BF. Healthy

PS C:\WINDOWS\System32> |
```

```
PS C:\WINDOWS\System32> Get-Partition

DiskPath: \\?\scsi#disk&ven_nvme&prod_samsung_mzvl4512#5&11b59e9b&0&0000000#{53f56307-b6bf-11d0-94f2-00a0c91efb8b}

PartitionNumber DriveLetter Offset Size Type
-----
1 1048576 200 MB System
2 210763776 16 MB Reserved
3 C 227540992 475.21 GB Basic
4 5104855... 812 MB Recovery
5 5113380... 735 MB Recovery

PS C:\WINDOWS\System32>
```

```
PS C:\WINDOWS\System32> Get-Partition

DiskPath: \\?\scsi#disk&ven_nvme&prod_samsung_mzvl4512#5&11b59e9b&0&0000000#{53f56307-b6bf-11d0-94f2-00a0c91efb8b}

PartitionNumber DriveLetter Offset Size Type
-----
1 1048576 200 MB System
2 210763776 16 MB Reserved
3 C 227540992 475.21 GB Basic
4 5104855... 812 MB Recovery
5 5113380... 735 MB Recovery

PS C:\WINDOWS\System32> Get-Volume

DriveLetter FriendlyName FileSystemType DriveType HealthStatus
-----
C NTFS Fixed Healthy
NTFS Fixed Healthy
NTFS Fixed Healthy

PS C:\WINDOWS\System32>
```

To understand how digital artifacts are generated and preserved, system operations are mapped across five core layers:

LAYER / SOURCE	EXAMPLE EVIDENCE	WHERE IT MAY BE FOUND	WHY IT MATTERS
USER LAYER	Typed keyboard input, desktop wallpaper changes, or window focus history	User Assist registry keys, ShellBags, or Recent Items folders	Directly associates specific human actions with interface interactions.
APPLICATION LAYER	Browser download history, locally saved chat transcripts, or email attachments	Application-specific SQLite databases, local cache folders (%AppData%)	Details the specific software tools utilized to manipulate or transmit data.
OS LAYER	Service start/stop times, driver installation records, or power state transitions	System Event Logs (.evtx), Windows Registry (SYSTEM hive)	Provides the system-wide timeline of administrative and background operations.
DISK LAYER	File slack space, Volume Shadow Copies (VSC), or hidden partition data	Raw physical sectors, Master File Table (MFT), or unallocated clusters	Facilitates recovery of deleted data and low-level forensic reconstruction.
MEMORY/PROCESS LAYER	Decrypted passwords in cleartext, active network socket states, or injected malicious DLLs	Pagefile.sys, hiberfil.sys, or captured volatile RAM dumps	Captures state-dependent artifacts that vanish upon system power-off.

Evidence frequently exists in more than one layer because user activity propagates across the entire architecture (User Activities = Applications + Operating System + Disk/Memory). For instance, initiating an action starts at the user interface, triggers application processing, generates operating system kernel and event logs, leaves persistent entries on the disk file system, and maintains temporary states in volatile memory. Correlating these artifacts across layers allows investigators to build reliable, verifiable timelines.

Part B - Process Observation and Management

To observe running system processes and understand system activity from both command-line and graphical interfaces, live process enumeration commands (tasklist) and Windows Task Manager were utilized.

Process enumeration was conducted via PowerShell to output active processes, Process IDs (PIDs), session names, and memory utilization metrics.

```
PS C:\WINDOWS\System32> tasklist
```

Image Name	PID	Session Name	Session#	Mem Usage
System Idle Process	0	Services	0	8 K
System	4	Services	0	4,448 K
Secure System	140	Services	0	65,232 K
Registry	184	Services	0	31,680 K
smss.exe	688	Services	0	1,472 K
csrss.exe	1096	Services	0	7,312 K
wininit.exe	1252	Services	0	9,580 K
services.exe	1380	Services	0	22,552 K
LsaIso.exe	1424	Services	0	4,260 K
lsass.exe	1432	Services	0	36,736 K
svchost.exe	1580	Services	0	72,528 K
fontdrvhost.exe	1604	Services	0	4,504 K
svchost.exe	1724	Services	0	23,948 K
svchost.exe	1780	Services	0	11,836 K
svchost.exe	1072	Services	0	6,512 K
svchost.exe	1168	Services	0	29,792 K
svchost.exe	2124	Services	0	19,244 K
svchost.exe	2128	Services	0	19,032 K
svchost.exe	2136	Services	0	8,396 K
svchost.exe	2132	Services	0	7,912 K

Simultaneously, the graphical interface view was captured using Task Manager to display active applications and background services categorized with resource consumption.

Name	Status	6% CPU	60% Memory	3% Disk	0% Network
Apps (6)					
> Brave Browser (18)		0.2%	801.6 MB	0.1 MB/s	0.1 Mbps
> Microsoft Word (2)		0%	133.3 MB	0 MB/s	0 Mbps
> Task Manager		1.9%	75.9 MB	0 MB/s	0 Mbps
> Terminal (2)		0%	11.6 MB	0 MB/s	0 Mbps
> VMware Workstation (32 bit) (2)		0%	3.6 MB	0 MB/s	0 Mbps
> Windows Explorer		0%	115.2 MB	0 MB/s	0 Mbps
Background processes (86)					
> AMD Crash Defender Service		0%	0.5 MB	0 MB/s	0 Mbps
> AMD External Events Client M...		0%	1.2 MB	0 MB/s	0 Mbps
> AMD External Events Service		0%	0.6 MB	0 MB/s	0 Mbps

Based on the process captures, several foundational Windows processes maintain system stability and operation:

- **System Idle Process (PID 0):** Represents the percentage of unused CPU capacity. High memory usage is negligible here, as its primary metric tracks processor downtime.
- **smss.exe (Session Manager Subsystem):** Responsible for initializing the user session, creating environment variables, and starting subsystems like csrss.exe.

- **csrss.exe (Client/Server Runtime Subsystem):** Manages graphical instruction sets, console windows, and thread creation.
- **wininit.exe (Windows Initialization Process):** Starts core background services, the Local Security Authority Process (lsass.exe), and the Service Control Manager (services.exe).
- **services.exe:** Manages the starting, stopping, and interaction of background Windows services.
- **lsass.exe (Local Security Authority Subsystem Service):** Enforces security policies, handles user logins, and manages password changes and access tokens.
- **svchost.exe (Service Host):** A generic host process container used to run multiple Windows services from DLL files, reducing resource overhead.

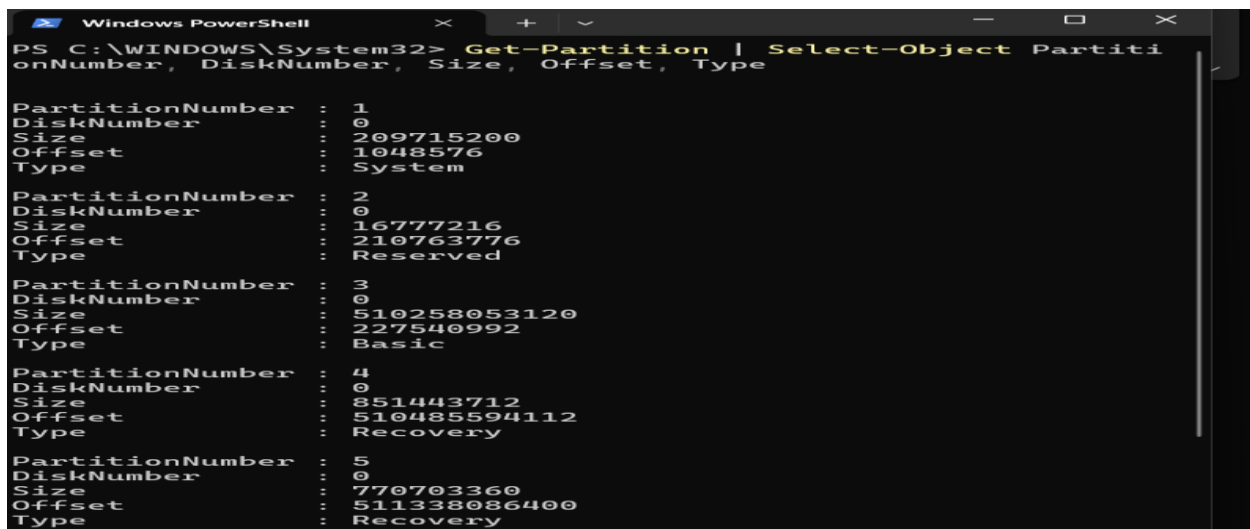
Difference between Program vs. Process vs. Thread

A program is just a static file sitting on your hard drive, a process is that program actually running in memory with its own resources, and a thread is the smaller path of execution inside that process doing the actual work.

Importance in Malware Analysis and Live Forensic Triage

Process information is useful during live triage because it instantly reveals active malicious activity, hidden injections, or running payloads that would otherwise remain invisible while sitting dormant on the disk.

Part C – Disk Sector and Disk Size Calculation



```

PS C:\WINDOWS\System32> Get-Partition | Select-Object PartitionNumber, DiskNumber, Size, Offset, Type

PartitionNumber : 1
DiskNumber      : 0
Size           : 209715200
Offset         : 1048576
Type           : System

PartitionNumber : 2
DiskNumber      : 0
Size           : 16777216
Offset         : 210763776
Type           : Reserved

PartitionNumber : 3
DiskNumber      : 0
Size           : 510258053120
Offset         : 227540992
Type           : Basic

PartitionNumber : 4
DiskNumber      : 0
Size           : 851443712
Offset         : 510485594112
Type           : Recovery

PartitionNumber : 5
DiskNumber      : 0
Size           : 770703360
Offset         : 511338086400
Type           : Recovery

```

(PowerShell output of Get-Partition showing Partition 1 size of 209715200 bytes and Partition 3 size of 510258053120 bytes)

C2. Calculate Your Own Partition Size

Using the partition size values in bytes extracted directly from your PowerShell output, here are the step-by-step calculations for Partition 1 and Partition 3 following the required format:

Partition 1 (System Partition — Size: 209,715,200 bytes)

ITEM	FORMULA / VALUE	ANSWER
TOTAL BYTES	Given partition size in bytes	209,715,200 bytes
KILOBYTES	$209,715,200/1024$	204,800 KB
MEGABYTES	$204,800/1024$	200 MB
GIGABYTES	$200/1024$	0.1953 GB

Partition 3 (Basic OS Partition — Size: 510,258,053,120 bytes)

ITEM	FORMULA / VALUE	ANSWER
TOTAL BYTES	Given partition size in bytes	510,258,053,120 bytes
KILOBYTES	$510,258,053,120/1024$	498,298,880 KB
MEGABYTES	$498,298,880/1024$	486,620 MB
GIGABYTES	$486,620/1024$	475.21 GB

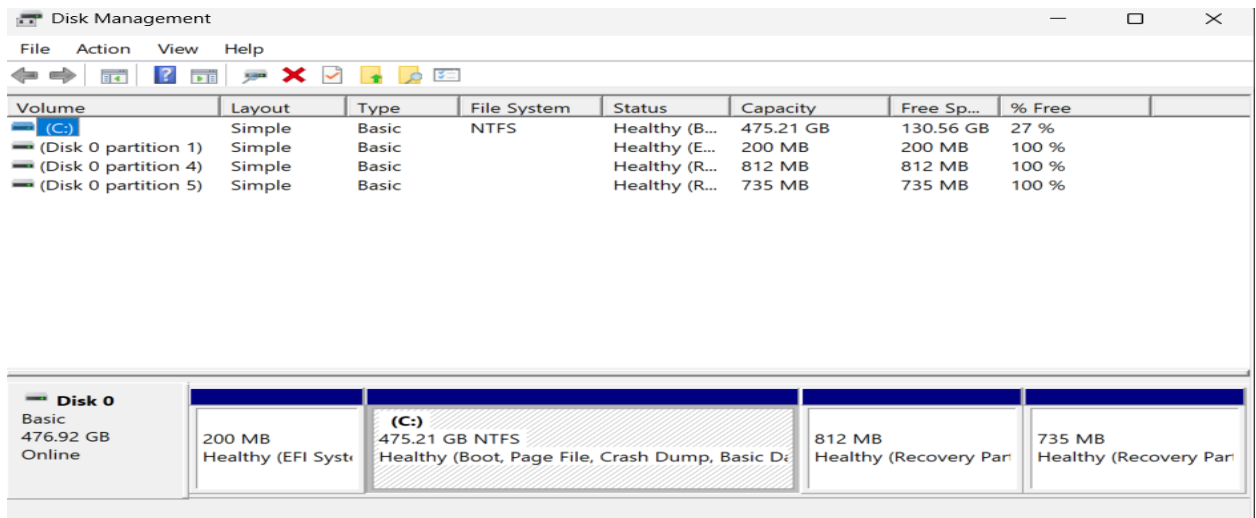
Part D – Safe Partition Table Inspection

Windows Method: Disk Management, PowerShell, and DiskPart Inspection

This section documents the safe, read-only inspection of the Windows partition table and disk layout using native Windows utilities (diskmgmt.msc, PowerShell, and DiskPart) without modifying the underlying disk.

Step 1: Disk Management Layout

1. Pressed Windows + R, typed diskmgmt.msc, and pressed **Enter** to open Disk Management.
2. Verified the disk layout, volume structure, file systems, and partition health status. Disk 0 is initialized as a **GPT (GUID Partition Table)** disk with a total capacity of 476.92 GB containing an EFI System Partition, the main OS volume (C:), a Microsoft Reserved partition, and recovery partitions.



Step 2: DiskPart Safe Listing Commands

1. Opened Command Prompt as an administrator and launched the interactive utility via diskpart.
2. Executed safe, read-only listing commands (list disk, select disk 0, list partition) to verify the partition types and offsets.

```
DISKPART> list disk

Disk ###  Status       Size       Free       Dyn  Gpt
-----  -
Disk 0    Online       476 GB     2048 KB

```

```
DISKPART> select disk 0
Disk 0 is now the selected disk.

DISKPART> list partition

Partition ##  Type         Size       Offset
-----  -
Partition 1   System       200 MB     1024 KB
Partition 2   Reserved     16 MB      201 MB
Partition 3   Primary      475 GB     217 MB
Partition 4   Recovery     812 MB     475 GB
Partition 5   Recovery     735 MB     476 GB

```

D2. Linux / Kali Method: Read-Only Partition Table Inspection

This section documents the safe, read-only inspection of disk layouts, block devices, partition tables, and mount points inside a Linux/Kali environment using standard diagnostic tools without writing changes (mkfs, fdisk writes, parted writes, or dd).

Step 1: Block Device Layout (lsblk)

- Ran `lsblk -o NAME,SIZE,TYPE,FSTYPE,MOUNTPOINT` to view the hierarchical relationship of block devices and their file systems.

Step 2: Partition Table Inspection (fdisk & parted)

- Executed `sudo fdisk -l` to inspect the disk label type, sector sizes, and raw partition entry data.
- Executed `sudo parted -l` to view logical and physical sector configurations alongside partition table types (such as msdos or mac).

Step 3: File System Attributes & Disk Space (blkid & df)

- Ran `blkid` to retrieve universally unique identifiers (UUIDs) and block device attributes safely in read-only mode.
- Ran `df -h` to examine mounted file system disk space utilization in human-readable formats.

```
(garba@kali)-[~]
└─$ lsblk -o NAME,SIZE,TYPE,FSTYPE,MOUNTPOINT
NAME        SIZE TYPE FSTYPE MOUNTPOINT
sda          45G disk
└─sda1       43G part ext4    /
└─sda2       1K part
└─sda5       2G part swap    [SWAP]
sr0         4.4G rom iso9660
```

```
(garba@kali)-[~]
└─$ sudo fdisk -l
[sudo] password for garba:
Disk /dev/sda: 45 GiB, 48318382080 bytes, 94371840 sectors
Disk model: VMware Virtual S
Units: sectors of 1 * 512 = 512 bytes
Sector size (logical/physical): 512 bytes / 512 bytes
I/O size (minimum/optimal): 512 bytes / 512 bytes
Disklabel type: dos
Disk identifier: 0xf75cda62

Device      Boot    Start        End    Sectors    Size Id Type
/dev/sda1   *          2048    90179583    90177536    43G 83 Linux
/dev/sda2           90181630  94369791    4188162      2G  f W95 Ext'd (LBA)
/dev/sda5           90181632  94369791    4188160      2G 82 Linux swap / Solaris
```

```
(garba@kali)-[~]
└─$ sudo parted -l
Model: VMware, VMware Virtual S (scsi)
Disk /dev/sda: 48.3GB
Sector size (logical/physical): 512B/512B
Partition Table: msdos
Disk Flags:

Number  Start   End     Size    Type     File system  Flags
 1      1049kB  46.2GB  46.2GB  primary  ext4         boot
 2      46.2GB  48.3GB  2144MB  extended
 5      46.2GB  48.3GB  2144MB  logical  linux-swap(v1) swap

Warning: Unable to open /dev/sr0 read-write (Read-only file system). /dev/sr0
has been opened read-only.
Model: NECVMWar VMware SATA CD01 (scsi)
Disk /dev/sr0: 4733MB
Sector size (logical/physical): 2048B/2048B
Partition Table: mac
Disk Flags:

Number  Start   End     Size    File system  Name  Flags
 1      2048B   6143B   4096B   Apple        Apple
 2      11.1MB  14.7MB  3670kB   EFI          EFI
```

```
(garba@kali)-[~]
$ blkid
/dev/sr0: BLOCK_SIZE="2048" UUID="2025-12-03-01-17-40-00" LABEL="Kali Linux amd64 1" TYPE="iso9660" PTUUID="0013fa64" PTTYPE="dos"

(garba@kali)-[~]
$ df -h
Filesystem      Size  Used Avail Use% Mounted on
udev            861M     0  861M   0% /dev
tmpfs           194M  1.2M  193M   1% /run
/dev/sda1        43G   28G   13G  70% /
tmpfs           967M   4.0K  967M   1% /dev/shm
none            1.0M     0   1.0M   0% /run/credentials/systemd-journald.service
tmpfs           967M   72K  967M   1% /tmp
none            1.0M     0   1.0M   0% /run/credentials/getty@tty1.service
tmpfs           194M  108K  194M   1% /run/user/1000

(garba@kali)-[~]
$
```

Part E - MBR, GPT, and Boot Process Explanation

E1. System Boot Process Breakdown

When a computer starts up, it follows a structured initialization sequence before handing control over to the user. Each phase plays a vital role during digital investigations:

Stage	What Happens	Forensic Relevance
Power-on and POST	The machine draws power, and the Power-On Self-Test verifies core components like RAM, CPU, and storage.	Hardware-level tampering or physical faults can hinder data extraction or evidence collection.
BIOS/UEFI firmware	The motherboard firmware runs, checking non-volatile memory for configuration settings and boot priority orders.	Identifying whether a machine booted via Legacy BIOS or UEFI explains the disk partition structure and boot artifacts found later.
MBR/GPT or EFI boot	The firmware scans the storage drive to locate the primary boot records or EFI system partition files.	Boot records retain configuration artifacts that can reveal unauthorized modifications or alternative boot configurations.
Boot loader/manager	Software like Windows Boot Manager or GRUB loads into memory to initialize OS files or offer a multi-boot menu.	Examining this stage can uncover hidden operating systems, dual-boot setups, or customized boot payloads.
Kernel and OS control	The core operating system kernel takes over, launching system drivers, services, and user authentication.	This is where persistent system logs, event histories, and user activity artifacts begin accumulating on the disk.

Firmware Identification for This System

Based on evidence gathered during our disk inspections (such as the presence of an EFI system partition and GPT formatting in Disk Management), this system utilizes modern UEFI firmware rather than a legacy BIOS setup.

E2. Structural Comparison: MBR vs. GPT

<i>Feature</i>	<i>MBR (Master Boot Record)</i>	<i>GPT (GUID Partition Table)</i>
<i>Design Origin</i>	Developed in the early 1980s alongside legacy BIOS architectures.	A modern partitioning standard introduced to replace MBR alongside UEFI.
<i>Capacity Limit</i>	Strictly restricted to handling drives up to roughly 2 TB in size.	Massively scalable, supporting storage capacities up to approximately 9.4 ZB.
<i>Table Storage</i>	Stores partition data and boot code directly inside the very first sector of the drive.	Distributes structural data using primary and secondary (backup) headers across the disk.
<i>Forensic Challenges</i>	Because vital details sit entirely in sector 0, minor corruption can cause total partition loss.	Features built-in redundancy via backup structures, making validation and recovery significantly safer.

Part F - Logical Disk and Volume Enumeration (Linux / Kali Environment)

Executed Commands and Utility Overview

To inspect mounted file systems, block storage hierarchies, and disk utilization safely in read-only mode, the following commands were run:

- `lsblk -f`: Lists block devices alongside their allocated file systems (ext4, swap, iso9660), version labels, UUIDs, and mount points.
- `blkid`: Displays low-level block device attributes and universally unique identifiers.
- `df -Th`: Provides a human-readable summary of mounted file systems, types (such as ext4 and tmpfs), total sizes, used space, and usage percentages.

F2. Terminal Evidence & Output Captures

- **Block Device File System Mapping (lsblk -f):**

```
(garba@kali)-[~]
$ lsblk -f
NAME        FSTYPE FSVER    LABEL              UUID                                 FSAVAIL FSUSE% MOUNTPOINTS
├─sda
│ ├─sda1    ext4    1.0              6fc8634a-03b7-4f32-9e68-da055b7dd141  12.3G   66% /
│ └─sda2
└─sda5      swap    1              ee9cb78f-aff2-4cd5-9665-ad7b05343131      [SWAP]
sr0         iso9660 Joliet Extension Kali Linux amd64 1 2025-12-03-01-17-40-00
```

- **Block Device Attribute Identification (blkid):**

```
(garba@kali)-[~]
$ lsblk -f
NAME        FSTYPE FSVER    LABEL              UUID                                 FSAVAIL FSUSE% MOUNTPOINTS
├─sda
│ ├─sda1    ext4    1.0              6fc8634a-03b7-4f32-9e68-da055b7dd141  12.3G   66% /
│ └─sda2
└─sda5      swap    1              ee9cb78f-aff2-4cd5-9665-ad7b05343131      [SWAP]
sr0         iso9660 Joliet Extension Kali Linux amd64 1 2025-12-03-01-17-40-00

(garba@kali)-[~]
$ blkid
/dev/sr0: BLOCK_SIZE="2048" UUID="2025-12-03-01-17-40-00" LABEL="Kali Linux amd64 1" TYPE="iso9660" PTUUID="0013fa64" PTTYPE="dos"

(garba@kali)-[~]
$
```

- **File System Disk Space Utilization (df -Th):**

```
(garba@kali)-[~]
$ blkid
/dev/sr0: BLOCK_SIZE="2048" UUID="2025-12-03-01-17-40-00" LABEL="Kali Linux amd64 1" TYPE="iso9660" PTUUID="0013fa64" PTTYPE="dos"

(garba@kali)-[~]
$ df -Th
Filesystem      Type      Size  Used Avail Use% Mounted on
udev            devtmpfs  861M   0    861M   0% /dev
tmpfs           tmpfs     194M  1.2M  193M   1% /run
/dev/sda1       ext4      43G   28G   13G   70% /
tmpfs           tmpfs     967M  4.0K  967M   1% /dev/shm
none            tmpfs     1.0M   0    1.0M   0% /run/credentials/systemd-journald.service
tmpfs           tmpfs     967M  8.0K  967M   1% /tmp
none            tmpfs     1.0M   0    1.0M   0% /run/credentials/getty@tty1.service
tmpfs           tmpfs     194M  108K  194M   1% /run/user/1000

(garba@kali)-[~]
$
```